



INFIX TO PREFIX (using Stacks)



RECAP

Infix: $A \times B$
Prefix: $\times A B$
Postfix: $A B \times$

In prefix, the operator comes before operands
In postfix, the operator comes after operands

Remember: The order of operands stays the same, only the position of the operator changes.

OPERATOR PRECEDENCE & ASSOCIATIVITY

When we evaluate expressions, operators don't all have the same priority.

Operator	Meaning	Precedence	Associativity
$\wedge$	Exponent	Highest	Right - Left
$\times / \%$	Multiply / Divide	Medium	Left - Right
$+-$	Add / Subtract	Lowest	Left - Right

Example: $A - B \times C$

If evaluation order is to be understood, it becomes $(A - B) \times C$ because $\times$ and $-$ are left associative.

STEPS TO FOLLOW:

Instead of writing a completely new algorithm, we reuse infix to postfix logic.

- Reverse the infix expression
- Swap brackets ('(' and ')')
- Convert the expression to postfix
- Reverse the result

That gives the prefix expression.

EXAMPLE OF THE FLOW

Given Expression is $(A+B \times C)$

Reverse: $)CB+A(+$
Swap brackets: $(CB+A)$
Postfix: $CB+A)$
Reverse: $(+AB)C$

Prefix result obtained

Other operators are left associative, so evaluation happens left to right.
BUT
Exponent operator $\wedge$ is right associative.
So evaluation happens right to left.
 $A \wedge B \times C$ becomes $A \wedge (B \times C)$

NOT $(A \wedge B) \times C$

Important

Why convert Infix Expressions?

Infix expressions are easy for humans to read, but they are slightly difficult for computers to evaluate because precedence rules must be checked constantly.
To simplify evaluation, compilers convert infix expressions into prefix or postfix because both eliminate ambiguity.

Prefix expressions are useful because the operator appears first, which allows evaluation without worrying about parentheses or precedence during evaluation.

Prefix: $A \times B \times C$
Meaning directly: $A \times (B \times C)$

Main conversion logic

- ~ Stack is used to store operators temporarily.
- ~ Operands go directly to output.
- ~ Operators are pushed into stack until precedence rules allow them to be popped.

Why Stack?

Characteristics of code:

Time Complexity $O(n)$
Each character processed once.

Space Complexity $O(n)$
Stack stores operators.

Uses stack data structure
Maintains precedence rules
Handles parentheses correctly

DIFFERENCE

Infix - Postfix	Infix - Prefix
No reversing required	Expression must be reversed
Direct conversion	Reverse $\rightarrow$ Postfix $\rightarrow$ Reverse
Slightly simpler	Slightly more steps

Which one is preferred then?

Usually infix to postfix is preferred.
Reason:
Simpler algorithm
Easier evaluation
Used more commonly

DRY RUN:

Assume the given expression as: $(A-B/C) \times (A/K-L)$

Step 1: Reverse the expression $\rightarrow)L-K/A(*)C/B-A(+$

Step 2: Swap brackets $\rightarrow (L-K/A) \times (C/B-A)$

Step 3: Conversion to Postfix (using Stack table)

Symbol	Stack	Output
(	(	
L	(	L
-	(-	L
K	(-	LK
/	(- /	LK
A	(- /	LKA
)	(	LKA/-
*	*	LKA/-
(	* (	LKA/-
C	* (	LKA/-C
/	* /	LKA/-C
B	* /	LKA/-CB
-	* (-	LKA/-CB/
A	* (-	LKA/-CB/A
)	*	LKA/-CB/A-
end		LKA/-CB/A-*

Postfix obtained: $LKA/-CB/A-*$

Step 4: Now simply reverse it to get the final prefix expression

$*-A/BC-/AKL$

Summing up:

- Infix is human readable but harder for computers
- Prefix and postfix remove ambiguity
- Infix to prefix conversion uses reverse $\rightarrow$ postfix $\rightarrow$ reverse trick
- Stack is used to temporarily store operators
- Precedence decides operator priority
- Associativity decides evaluation order when precedence is same
- Exponent operator $\wedge$ is right associative, others are left associative

And finally:

"Operands go directly to output, operators wait in stack."